

Java™ e o Modelo de Objetos

Java Deployment Course: Aula 1



por Jorge H C Fernandes
(jorge@dimap.ufrn.br)
DIMAp-UFRN

Objetivos desta Aula



- ⌘ Introduzir conceitos básicos do modelo de objetos
- ⌘ Criar um pequeno programa em Java que demonstre a aplicação dos conceitos de orientação a objetos

Alcance dos Conceitos do Modelo de Objetos



Conceitos Básicos do Modelo de Objetos



Conceitos Básicos do Modelo de Objetos : Parte 1



⌘ Objetos

⌘ Atributos, Variáveis, Campos

⌘ Métodos

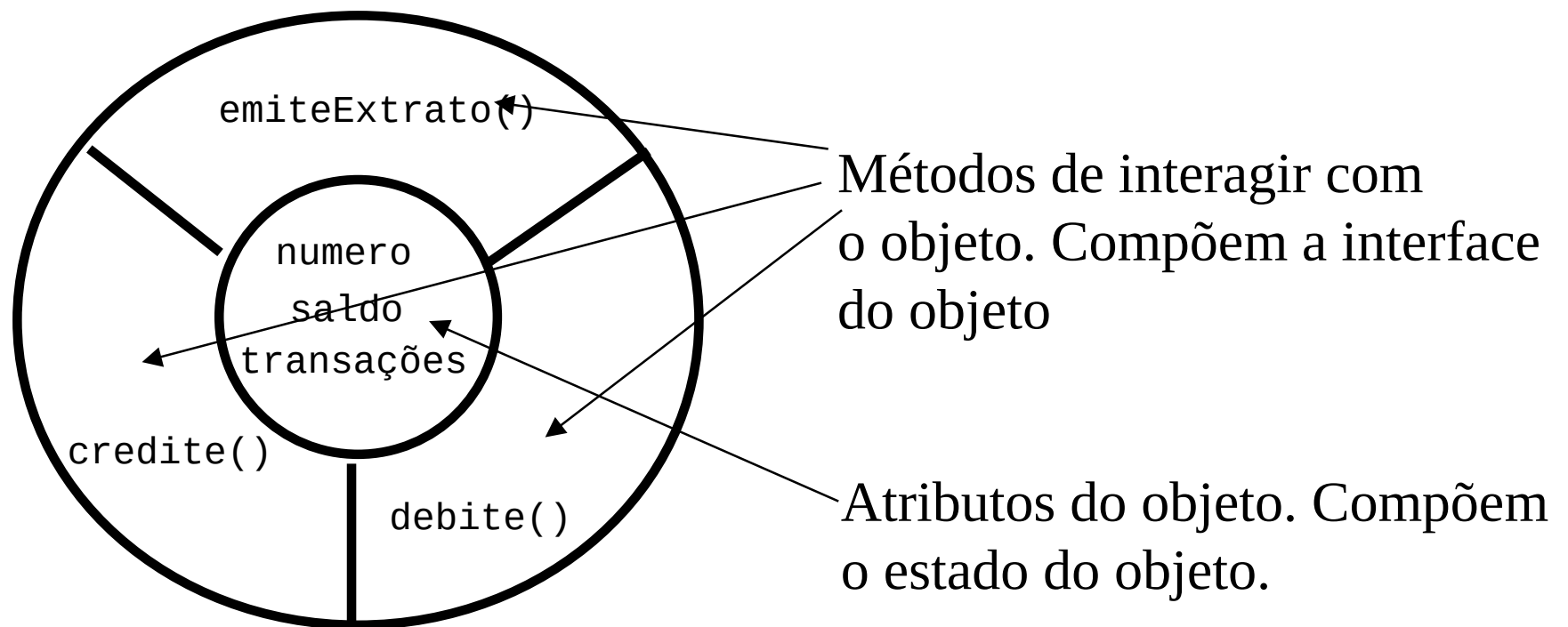
⌘ Mensagens

⌘ Classes

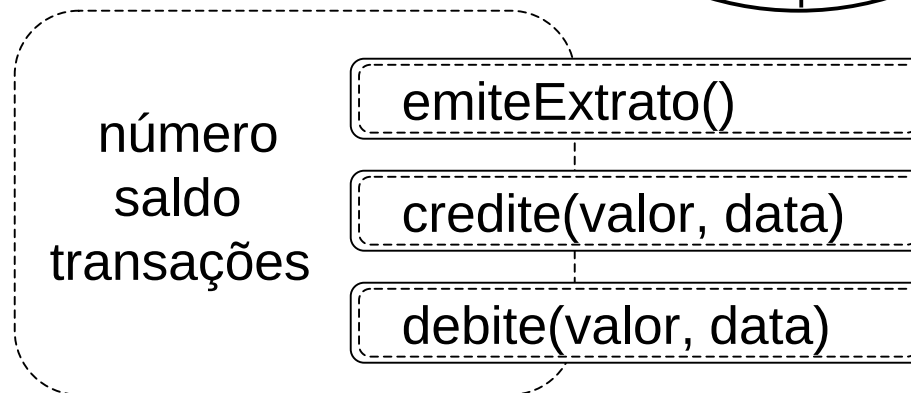
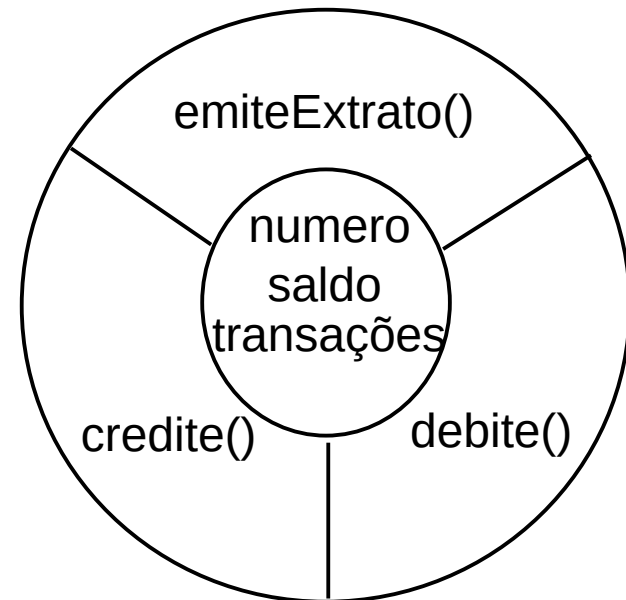
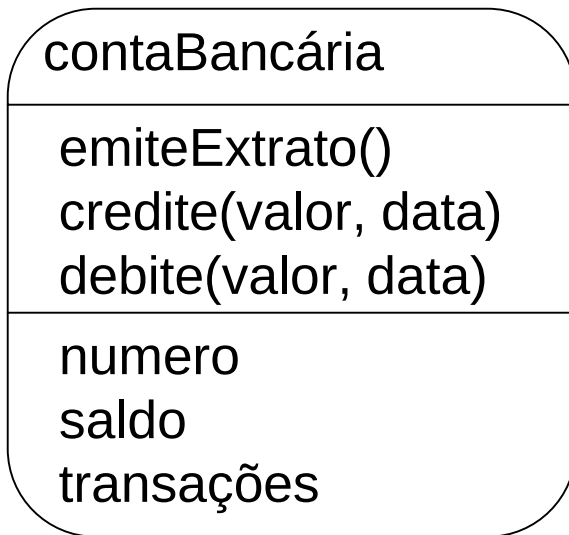
O Quê é um Objeto?

- ⌘ Um módulo de software bem definido, que está presente em um programa em execução
 - ☒ Apresenta um conjunto de atributos e métodos
- ⌘ Possui uma identidade e um comportamento bem definidos
- ⌘ Ocupa espaço na memória, assim como um processo (programa em execução)

Representação Gráfica de um Objeto



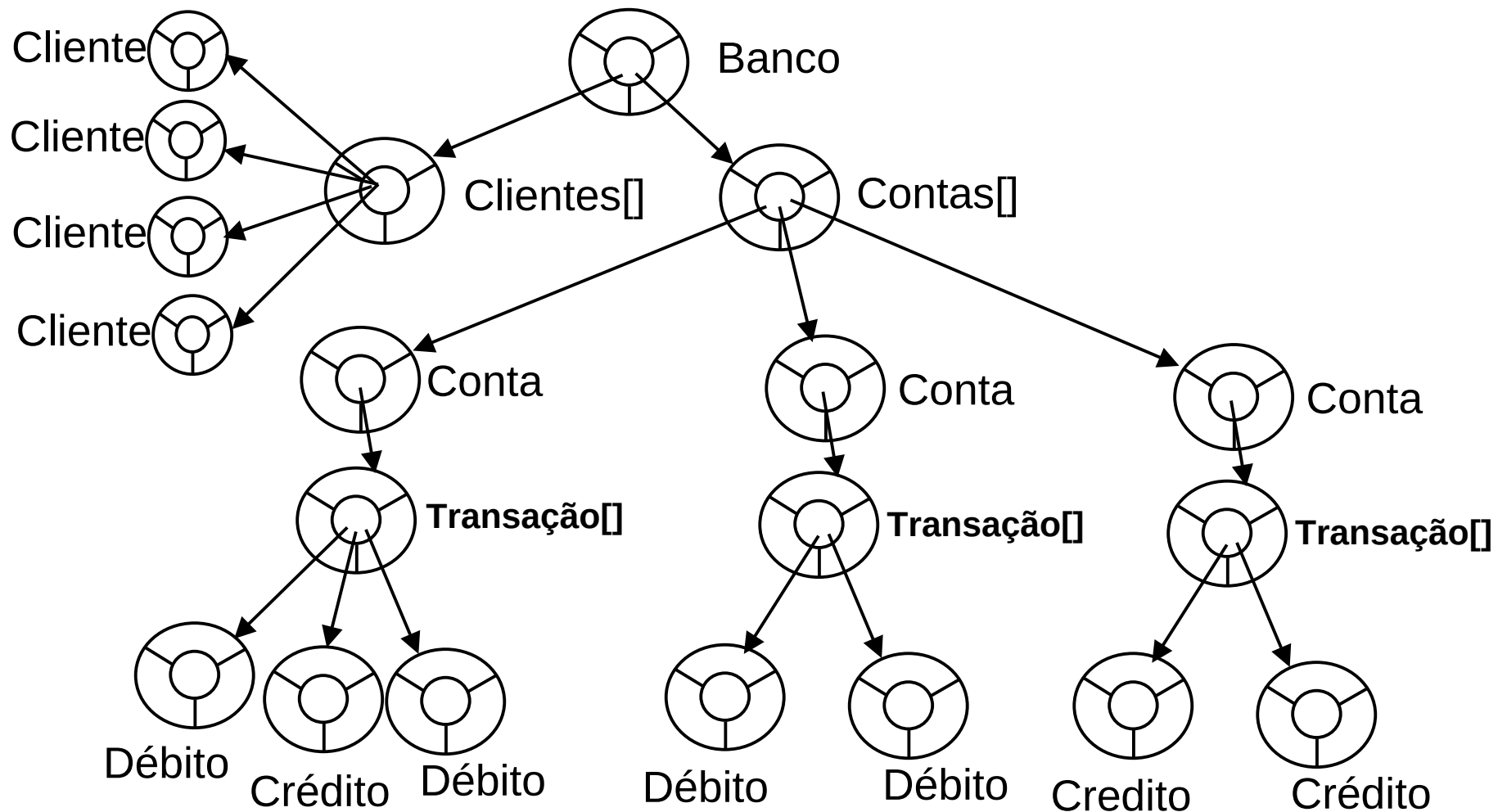
Outras Representações para Objetos



Atributos de um Objeto

- ⌘ Os atributos de um objeto são armazenados no interior destes, através de VARIÁVEIS ou CAMPOS
- ⌘ Os campos de um objeto AGREGAM
 - ☑ Valores primitivos (inteiros, bytes, etc) e,
 - ☑ Referências a outros objetos
- ⌘ O valor dos campos de um objeto em um determinado momento forma seu ESTADO

Objetos Agregando outros Objetos



Método



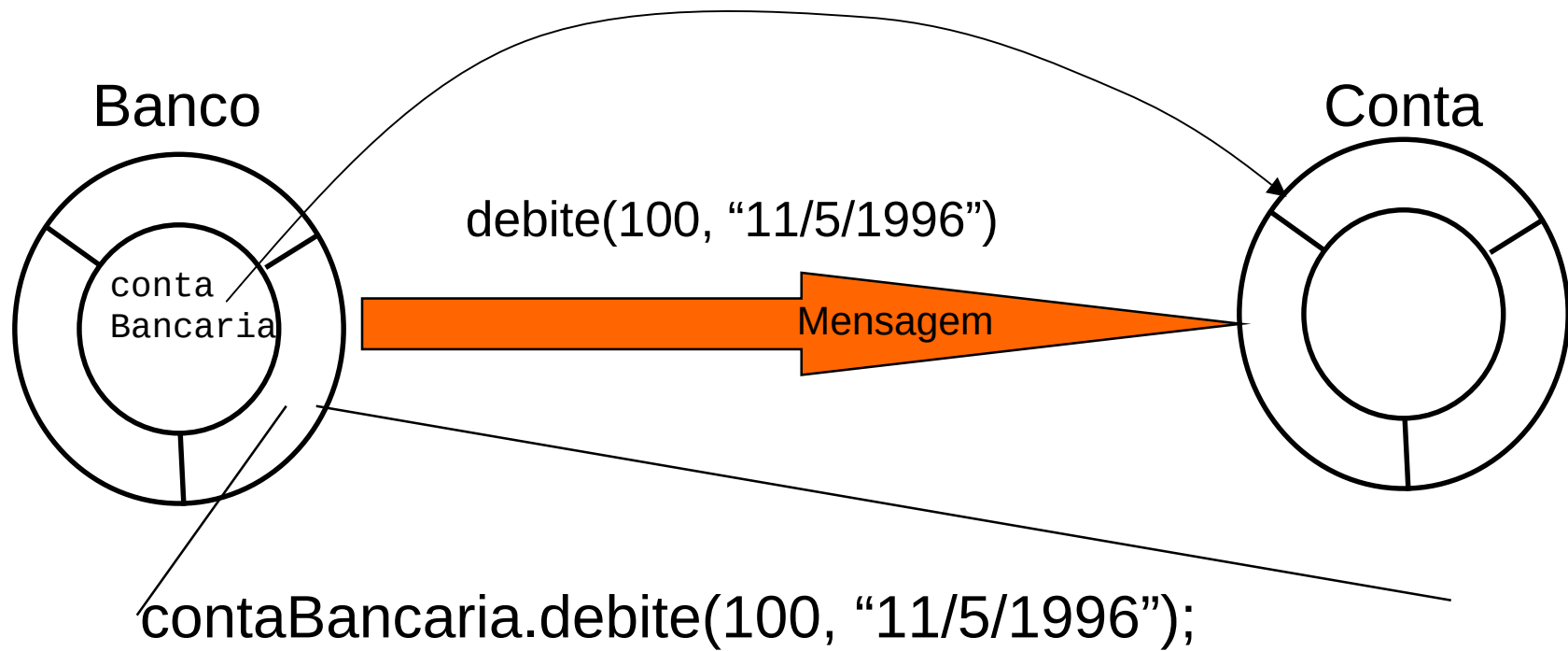
- ⌘ É a porta exterior através do qual se interage com o objeto
- ⌘ Um método é definido pela sua assinatura e pelo código que executa quando invocado

```
// O método debite descreve o que uma  
// conta bancária deve fazer quando debitando
```

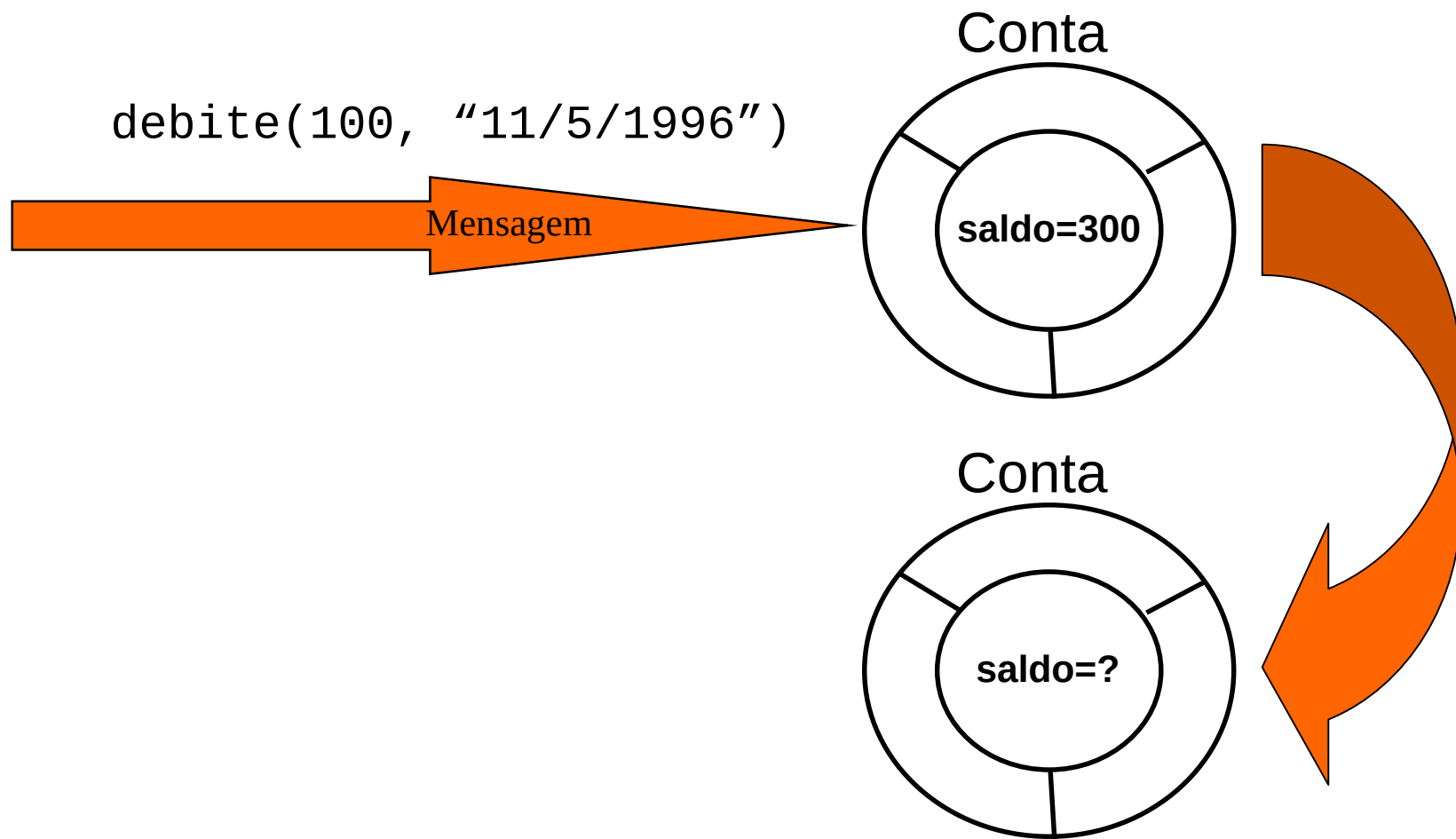
```
void debite(int valor, String data) {  
    saldo = saldo - valor;  
    ...  
}
```

Mensagem

⌘ Corresponde à invocação de método de um objeto (chamado) por outro objeto (chamador)



Qual o Saldo da Conta Após a Invocação do Método Credite?

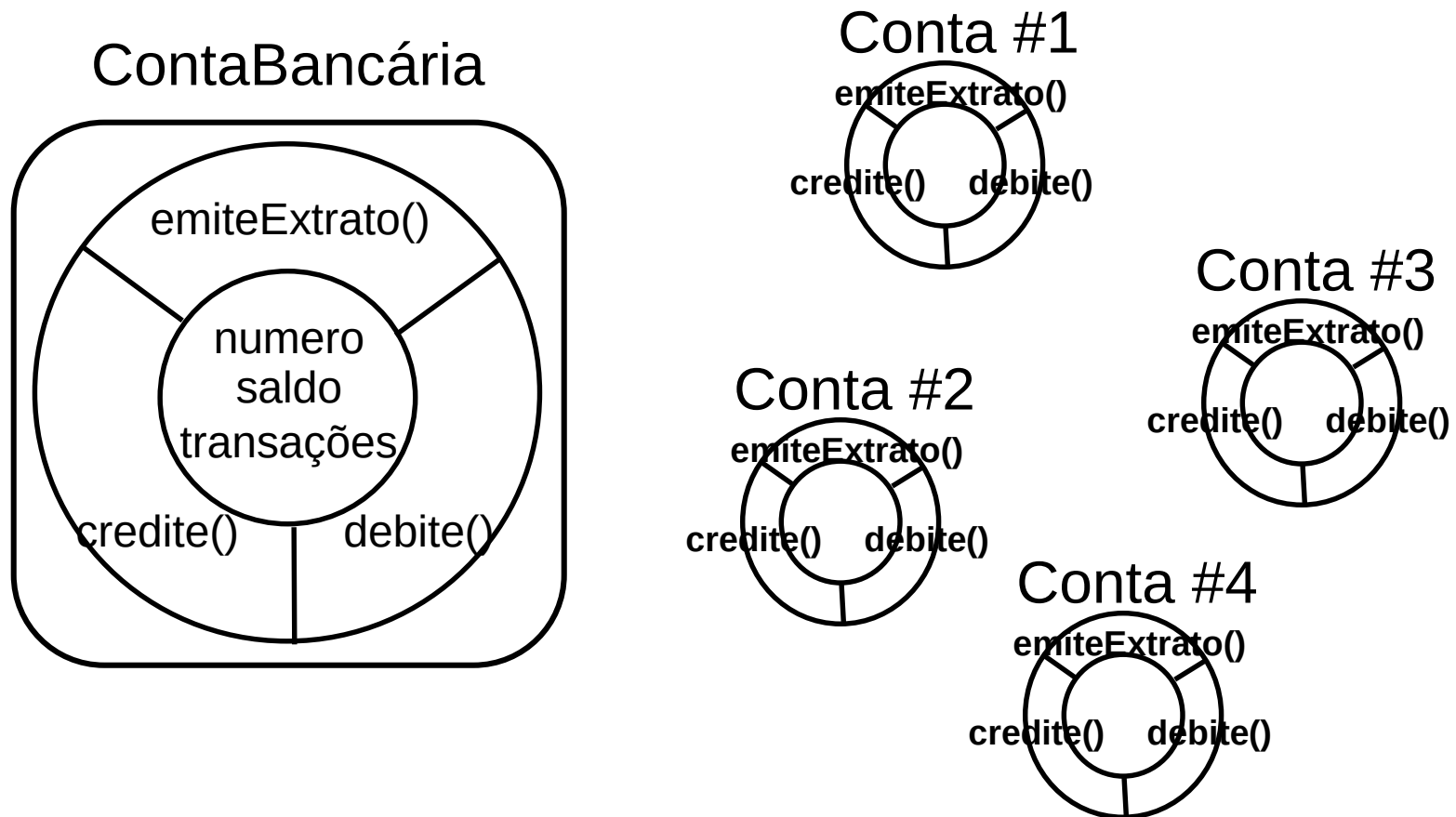


Classes



- ⌘ São o código que define o comportamento dos objetos
- ⌘ São moldes para criação de objetos
 - ☑ Muitos Objetos, Poucas Classes
- ⌘ A classe é responsável pela criação dos objetos

Representação Gráfica de uma Classe e Vários Objetos



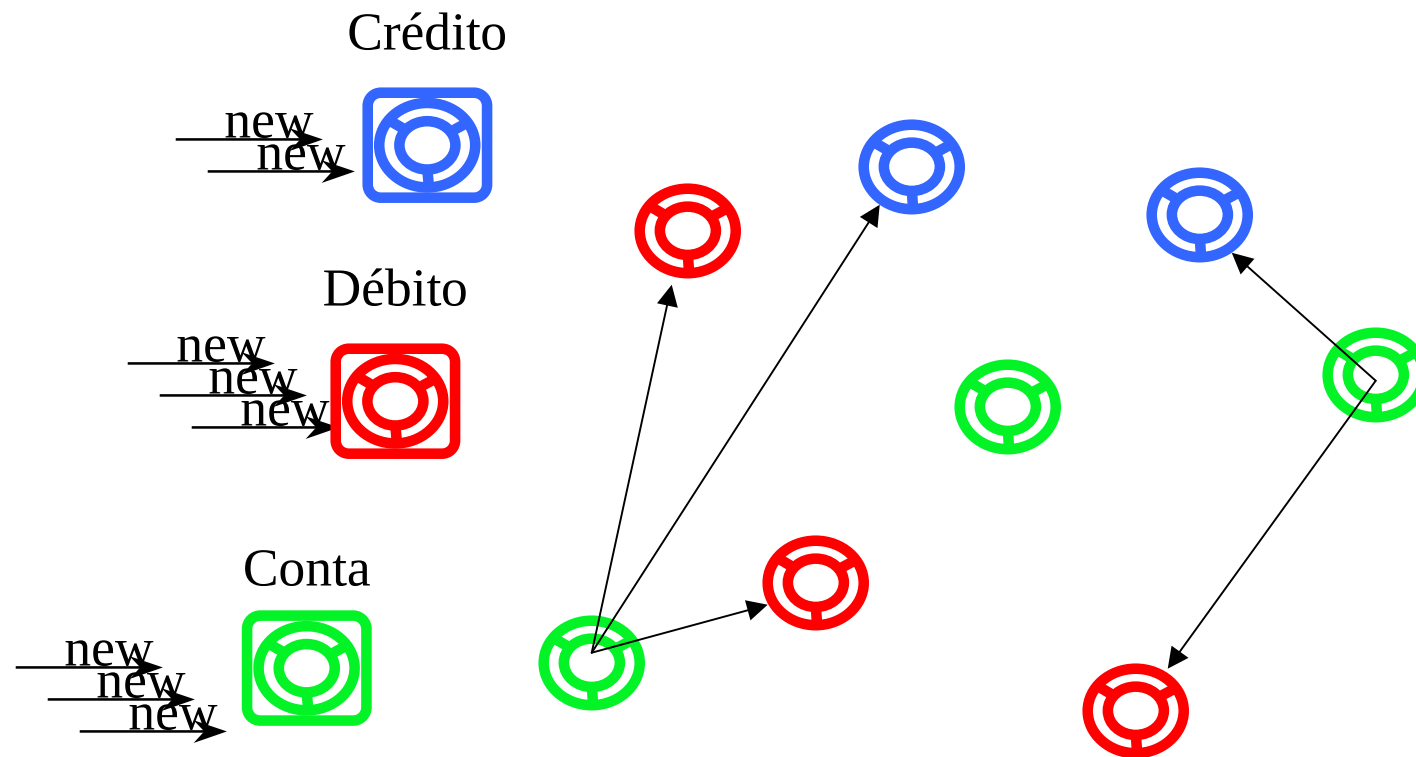
Outra Representações Gráficas para Classe e Objeto

Conta
emiteExtrato() credite(valor, data) debite(valor, data)
numero saldo transações

Conta
numero=1 saldo=500 transações

Conta
numero=2 saldo=500 transações

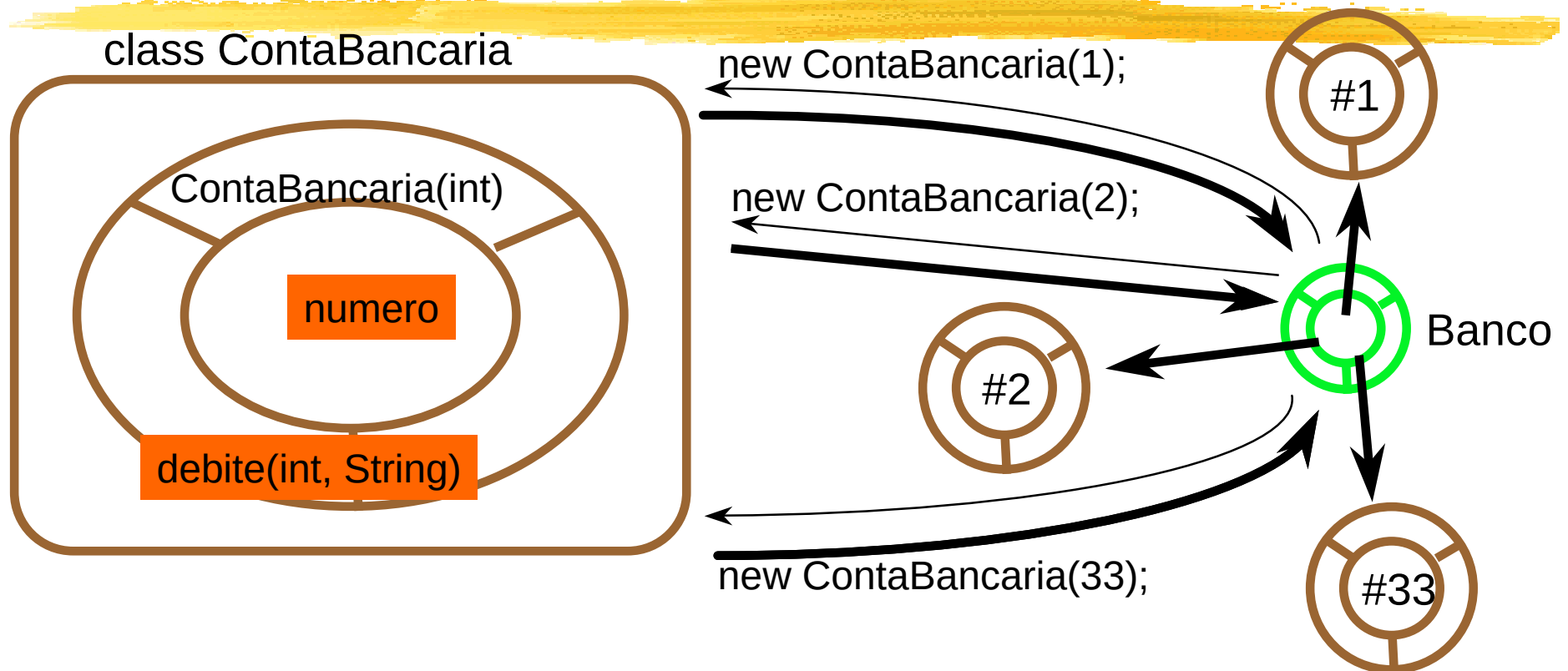
Criação de Algumas Contas e Transações



Definindo a Classe ContaBancária em Java

```
public class ContaBancaria {  
    private int numero;  
    private int saldo;  
    private Transacao[] transacoes;  
  
    // método construtor  
    public ContaBancaria(int numero) {...}  
  
    // métodos de instância  
    public void debite(int valor, String data) { ... }  
    public void credite(int valor, String data) { ... }  
    public void emiteExtrato() { ... }  
}
```

Criando Contas Bancárias



`ContaBancaria(int)`

`numero`

`debite(int, String)`

Método construtor

Atributo (variável de instância)

Método (de instância)

Criando Contas Bancárias

⌘ Três contas bancárias criadas dentro de um banco.

```
class Banco {  
    ContaBancaria conta1, conta2, conta33;  
    public void crieTresContas() {  
        conta1 = new ContaBancaria(1);  
        conta2 = new ContaBancaria(2);  
        conta3 = new ContaBancaria(33);  
    }  
    ...  
}
```

Exercício:

Encontrando Classes e Objetos



Onde Estão as Classes, Objetos, Métodos e Atributos?

⌘ De um sistema de

☑ Gestão de pessoal?

☑ Tributação?

☑ Controle de tráfego aéreo?

☑ Hospitalar?

☑ Automação fabril?

☑ etc

Exercício Simulado: Criação do Sistema de Controle Bancário



Controle de Conta Bancária

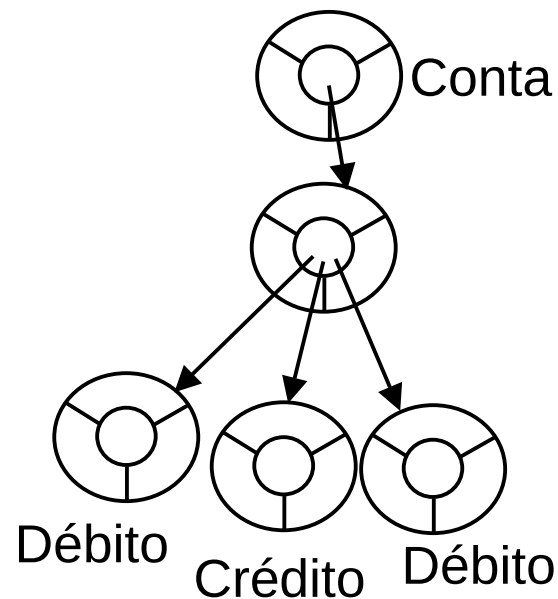
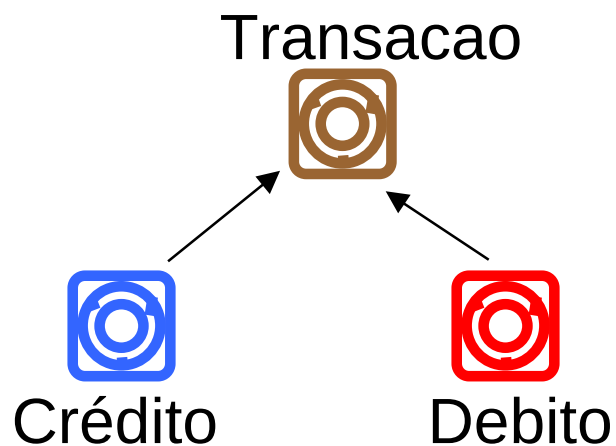
⌘ Definir classes

☐ ContaBancária, Transação, TransaçãoDébito e TransaçãoCredito

⌘ Elaborar um esquema polimórfico de emissão de extrato bancário que utilize as classes acima

⌘ Elaborar código que cria uma ContaBancária, realiza algumas transações de crédito e débito e emitir um extrato

Polimorfismo em uma Aplicação Bancária



Transação: Projeto



⌘ Estado formado pelo **valor** e **data**

⌘ Métodos:

- ☑ Construtor (valor e data)
- ☑ Emite linha de extrato (abstrato)
- ☑ Recupera valor da transação
- ☑ Recupera data da transação
- ☑ Indica qual a variação que a transação realiza sobre o saldo da conta (abstrato)

Transação: Implementação



- ⌘ Valor da transação armazenado como inteiro (`int`)
- ⌘ Data da transação armazenada como string (`String`)
- ⌘ Emissão de linha de extrato efetuada através da saída padrão (`System.out`)

Transação:

Esqueleto de Código



```
abstract class Transacao {  
    int valor;  
    String data;  
  
    Transacao(int valor, String data) {...}  
    int getValor(){...}  
    String getData() {...}  
  
    abstract int getVariação();  
    abstract void emiteLinhaExtrato();  
}
```

TransaçãoCrédito: Projeto e Implementação

- ⌘ Herda de transação
- ⌘ Implementa métodos abstratos de transação
 - ☑ Emite linha de extrato
 - ☑ Indica a variação **POSITIVA** que a transação realiza sobre o saldo da conta

TransaçãoCrédito: Esqueleto de Código

```
class TransacaoCredito extends Transacao {  
  
    TransacaoCredito(int valor, String data) {...}  
  
    int getVariação() {...}  
    void emiteLinhaExtrato() {...}  
}
```

Transação Débito: Projeto e Implementação

- ⌘ Herda de transação
- ⌘ Implementa métodos abstratos de transação
 - ☑ Emite linha de extrato
 - ☑ Indica a variação **NEGATIVA** que a transação realiza sobre o saldo da conta

TransaçãoDébito: Esqueleto de Código



```
class TransacaoDebito extends Transacao {  
  
    TransacaoDebito(int valor, String data) {...}  
  
    int getVariação() {...}  
    void emiteLinhaExtrato() {...}  
}
```


ContaBancária: Projeto



⌘ Estado formado pelo **número da conta**, **saldo anterior** e **transações** efetuadas

⌘ Métodos:

- ☑ Construtor (número da conta)
- ☑ Recupera número da conta
- ☑ Recupera saldo atual da conta
- ☑ Lança **débito** (valor e data)
- ☑ Lança **crédito** (valor e data)
- ☑ Emite extrato

Conta Bancária: Implementação



- ⌘ O saldo inicial da conta é sempre 0
- ⌘ Saldo armazenado como inteiro (`int`), representa o saldo antes de efetivar as transações (saldo anterior)
- ⌘ Transações são armazenadas em um array de tamanho fixo
- ⌘ Emissão de extrato efetuada através da saída padrão (`System.out.println()`)

ContaBancária:

Esqueleto de Código



```
class ContaBancaria {  
    int numero;  
    int saldoAnterior;  
    Transacao[] transacoes;  
  
    ContaBancaria(int numero) {...}  
    int getNumero(){...}  
    int getSaldoAtual() {...}  
    void debite(int valor, String data) {...}  
    void credite(int valor, String data) {...}  
    void emiteExtrato() {...}  
}
```

Criação de Contas e Transações



```
class ContaBancaria {  
    ..  
    public static void main(String[] args) {  
        ContaBancaria conta =  
            new ContaBancaria(...);  
        ..  
        conta.debite(20, "23/04/1999");  
        conta.credite(40, "26/04/1999");  
        ..  
        conta.emiteExtrato();  
        ..  
    }  
}
```

Exercício Simulado: Usando o JBuilder



⌘ Familiarizando-se com a interface

- ☑ Criação de projetos e classes

- ☑ Criação do HelloWorld

⌘ Criação do Sistema de Controle de Conta Bancária definido anteriormente